

UD01 - Fundamentos y diseño de interfaces de usuario

- [1. Introducción](#)
 - [1.1 Interfaces](#)
 - [1.1.1 Tipos de Interfaces según la Naturaleza de Interacción](#)
 - [1.1.2 Tipos de Interfaces según el Dispositivo Físico](#)
 - [1.2 El papel del prototipado](#)
- [2. Experiencia de Usuario \(UX\)](#)
- [3. Arquitectura de la información](#)

1. Introducción

Bienvenidos a la primera unidad del módulo de Desarrollo de Interfaces, en la cual se aborda el diseño de aplicaciones centrado en la experiencia del usuario. A partir de unas necesidades y requisitos previamente definidos, se desarrolla el proceso completo de prototipado para transformar dicha base funcional en una interfaz atractiva, funcional y accesible.

A lo largo de las siguientes etapas, empleando Figma como herramienta principal de trabajo, se aprende a estructurar la navegación, seleccionar los controles adecuados, aplicar criterios de legibilidad y estética, y validar el resultado final. El objetivo consiste en adquirir las bases de usabilidad y accesibilidad imprescindibles para el perfil de desarrollo, garantizando que las aplicaciones ofrezcan una experiencia fluida e intuitiva.

1.1 Interfaces

Una interfaz se define como el canal, medio o conjunto de elementos (tanto de hardware como de software) que permite la comunicación y el intercambio de información entre un usuario y un sistema informático. Su propósito fundamental es traducir las acciones del usuario en instrucciones comprensibles para la máquina y, a su vez, mostrar los resultados de forma clara y accesible.

1.1.1 Tipos de Interfaces según la Naturaleza de Interacción

Esta clasificación atiende a cómo se comunica el usuario con el sistema:

- **CLI (Command Line Interface - Interfaz de Línea de Comandos):** El usuario interactúa mediante comandos de texto escritos secuencialmente en una terminal o consola (por ejemplo, Bash, PowerShell). Requiere un conocimiento previo de la sintaxis, pero ofrece una gran potencia y eficiencia para tareas técnicas.
- **GUI (Graphical User Interface - Interfaz Gráfica de Usuario):** Utiliza elementos visuales y metafóricos (ventanas, iconos, botones, menús) manipulados habitualmente con un ratón, teclado o pantalla táctil. Es el estándar en la informática de consumo actual (por ejemplo, sistemas operativos de escritorio o aplicaciones móviles).
- **NUI (Natural User Interface - Interfaz Natural de Usuario):** Permite una interacción intuitiva mediante acciones cotidianas del ser humano, como

gestos corporales, movimientos o el reconocimiento espacial (por ejemplo, sistemas de control por gestos o tecnologías de realidad mixta).

- **VUI (Voice User Interface - Interfaz de Voz):** Basada en el procesamiento del lenguaje natural y la voz, permitiendo al usuario interactuar mediante comandos hablados (por ejemplo, asistentes virtuales como Alexa o Siri). Metodológicamente, la VUI se considera una especialización o subconjunto dentro de las interfaces de tipo NUI, enfocada específicamente en el canal auditivo y verbal.

1.1.2 Tipos de Interfaces según el Dispositivo Físico

Esta clasificación se centra en el soporte material y el factor de forma a través del cual se presenta la interfaz:

- **Dispositivos de Escritorio (Desktop):** Diseñados para pantallas grandes, operados principalmente mediante ratón y teclado. Permiten interfaces densas con múltiples ventanas, barras de herramientas y menús complejos.
- **Dispositivos Móviles (Smartphones y Tablets):** Enfocados en superficies táctiles de menor tamaño. Requieren diseños adaptativos (responsive), elementos interactivos sobredimensionados para los dedos y una organización vertical de la información.
- **Dispositivos Wearables (Tecnología Vestible):** Integrados en accesorios como relojes inteligentes o gafas de realidad aumentada. Exigen interfaces minimalistas, con pantallas reducidas y una fuerte dependencia de interacciones por voz o toques muy simples.
- **Sistemas Embebidos y Pantallas Especializadas:** Presentes en quioscos interactivos, paneles de control industrial, electrodomésticos inteligentes o pantallas de vehículos. Suelen requerir interfaces hiperespecializadas, orientadas a tareas muy concretas y con restricciones físicas particulares.

1.2 El papel del prototipado

En el ámbito del desarrollo de software, existe la tendencia errónea de comenzar a programar la interfaz de manera improvisada. Sin embargo, el diseño y el prototipado previo actúan como los planos de un edificio.

- Diseñar antes de programar mediante herramientas como Figma permite:

- Validar la lógica de negocio y los flujos de usuario antes de incurrir en costes de desarrollo o refactorización de código.
- Detectar problemas de accesibilidad y usabilidad de forma temprana.
- Alinear las expectativas entre los requisitos funcionales del sistema y la experiencia final que percibirá el usuario.

2. Experiencia de Usuario (UX)

En el proceso de diseño y desarrollo de una aplicación, la interfaz gráfica no debe entenderse meramente como una capa estética o un conjunto de pantallas decorativas, sino como el punto de contacto fundamental con el usuario. Para lograr que este contacto sea exitoso, la disciplina del desarrollo se apoya en la Experiencia de Usuario (UX) como paraguas global, destacando en él la usabilidad y la accesibilidad como los dos criterios más críticos e imprescindibles en el ámbito técnico de este módulo.

2.1 El enfoque global de la UX

La Experiencia de Usuario (UX o User Experience) engloba todas las percepciones, emociones y respuestas que experimenta una persona antes, durante y después de interactuar con un sistema informático. El concepto de UX funciona como un gran paraguas que no solo incluye el diseño visual, sino múltiples dimensiones que determinan el éxito de un producto digital:

- **Utilidad:** ¿El software resuelve un problema real del usuario? (Si una app hace algo que nadie necesita, la UX fracasa).
- **Deseabilidad:** El atractivo visual, la identidad de marca, el diseño emocional y la conexión que genera la interfaz.
- **Credibilidad:** La confianza y seguridad que transmite el sistema (por ejemplo, que una pasarela de pago aparente ser segura y fiable).
- **Validez y Accesibilidad:** Que el producto sea utilizable de forma universal y cumpla con los estándares legales y éticos.

Ejemplo

Una aplicación bancaria puede tener un diseño visual moderno y muy "deseable" (buena UX estética), pero si la navegación es confusa y no permite realizar una

transferencia de forma intuitiva, la experiencia global se arruina por completo. En el perfil de desarrollo de DAM, aunque el análisis profundo de negocio ya venga dado, comprender este ecosistema permite entender por qué se programa cada pantalla.

2.2 La usabilidad

La usabilidad es una cualidad que mide la facilidad con la que los usuarios pueden utilizar una herramienta interactiva para alcanzar sus objetivos de manera eficaz, eficiente y satisfactoria. Derivada directamente del diseño centrado en el usuario y situada en el núcleo de la UX, la usabilidad determina si una aplicación es intuitiva o si, por el contrario, genera frustración. A nivel industrial, este concepto se rige por normativas como la ISO 9241, y representa una de las prioridades técnicas más importantes al desarrollar interfaces.

2.2.1 Conceptos clave de usabilidad

Para valorar si una interfaz es usable, se suelen analizar dimensiones como:

- **Facilidad de aprendizaje (Learnability):** La rapidez con la que un usuario nuevo puede realizar tareas básicas la primera vez que se enfrenta al sistema.
- **Eficiencia operativa:** Una vez aprendida la interfaz, la velocidad y el nivel de productividad con el que se completan las tareas.
- **Memorabilidad:** La capacidad del usuario para recordar cómo se usa la aplicación tras un periodo prolongado sin utilizarla.
- **Tasa de errores:** La frecuencia con la que los usuarios cometen fallos, la gravedad de estos y la facilidad para recuperarse de ellos.
- **Satisfacción:** El grado de agrado y comodidad que transmite el uso de la aplicación.

Ejemplo

En un formulario de registro, un sistema usable avisa al instante si la contraseña no cumple los requisitos (mientras se escribe) en lugar de esperar a que el usuario pulse "Enviar", falle todo el registro y se borren los datos introducidos.

2.2.2 Los 10 Principios de Usabilidad de Jakob Nielsen

En el ámbito del diseño de interfaces, los 10 heurísticos o principios generales de usabilidad de Jakob Nielsen constituyen el marco teórico y práctico más extendido para evaluar y diseñar interfaces eficientes:

1. **Visibilidad del estado del sistema:** El sistema siempre debe mantener informado al usuario sobre lo que está ocurriendo, mediante retroalimentación visual en un tiempo razonable. (Ejemplo: Una barra de progreso o un icono de carga mientras se procesa un pago).
2. **Relación entre el sistema y el mundo real:** El sistema debe hablar el idioma del usuario, utilizando palabras, frases y conceptos familiares para este, en lugar de términos orientados puramente a la máquina. (Ejemplo: Utilizar el icono de una papelera de reciclaje en lugar del término técnico "delete_inode.log").
3. **Control y libertad del usuario:** Los usuarios suelen elegir las funciones del sistema por error y necesitan un "salida de emergencia" claramente marcada para abandonar el estado no deseado sin tener que pasar por una serie de pasos complejos. Se admiten las funciones de deshacer (undo) y rehacer (redo).
4. **Consistencia y estándares:** Los usuarios no deberían tener que averiguar si diferentes palabras, situaciones o acciones significan lo mismo. Se deben seguir las convenciones de la plataforma y del sector. (Ejemplo: Colocar siempre el botón de "Aceptar" en la misma posición relativa dentro de los cuadros de diálogo).
5. **Prevención de errores:** Mucho mejor que un buen mensaje de error es un diseño cuidadoso que previene la aparición de estos problemas de entrada. Se eliminan las condiciones propensas a errores o se solicita una confirmación antes de que el usuario ejecute la acción. (Ejemplo: Deshabilitar el botón de "Enviar formulario" hasta que todos los campos obligatorios estén correctamente rellenos).
6. **Reconocimiento antes que recuerdo:** Minimizar la carga cognitiva del usuario haciendo visibles los objetos, acciones y opciones. El usuario no debería tener que recordar información de una parte de la interfaz a otra. (Ejemplo: Mostrar un historial de búsquedas recientes para que el usuario pueda hacer clic en lugar de volver a escribirlo).
7. **Flexibilidad y eficiencia de uso:** Los atajos —ocultos para los usuarios novatos— pueden acelerar la interacción para el usuario experto, permitiendo que el sistema se adapte a ambos niveles de conocimiento. (Ejemplo: Combinaciones de teclas rápidas como Ctrl+C o menús personalizables).

8. **Estética y diseño minimalista:** Las diálogos y pantallas no deben contener información que sea irrelevante o poco frecuente. Cada unidad extra de información en un diálogo compite con las unidades de información relevante y disminuye su visibilidad relativa.
9. **Ayudar a los usuarios a reconocer, diagnosticar y recuperarse de errores:** Los mensajes de error deben expresarse en un lenguaje claro (sin códigos de error crípticos), indicar exactamente cuál es el problema y sugerir una solución constructiva. (Ejemplo: En lugar de "Error 404: NullPointerException", mostrar "No se ha encontrado el archivo solicitado. Compruebe la ruta e inténtelo de nuevo").
10. **Ayuda y documentación:** Aunque es mejor que el sistema pueda usarse sin documentación, puede ser necesario proveer ayuda y documentación. Esta debe ser fácil de buscar, estar enfocada en las tareas del usuario y enumerar los pasos concretos necesarios.

2.3 Accesibilidad

La accesibilidad complementa a la usabilidad asegurando que la aplicación pueda ser utilizada por el mayor número posible de personas, independientemente de sus capacidades físicas (motrices, visuales, auditivas), cognitivas o de las restricciones tecnológicas y ambientales del entorno. En el desarrollo de software actual, garantizar unos mínimos accesibles no constituye únicamente una buena práctica de ingeniería, sino que se ha convertido en un requisito técnico indispensable y, en muchos ámbitos, normativo.

Mientras que la usabilidad busca que un sistema sea eficiente y fácil de usar para el usuario general, la accesibilidad derriba las barreras que excluyen a personas con necesidades diversas.

2.3.1 El Marco Normativo y los Principios WCAG

El estándar internacional de referencia en accesibilidad digital se sustenta en las Pautas de Accesibilidad para el Contenido Web (WCAG), promovidas por el W3C. Aunque nacieron enfocadas en la web, sus directrices se aplican hoy de forma universal al diseño de cualquier interfaz de software interactiva.

El marco normativo se estructura sobre cuatro pilares fundamentales, conocidos por el acrónimo POUR:

1. **Perceptible (Perceivable):**

La información y los componentes de la interfaz deben presentarse de modo que los usuarios puedan percibirlos a través de sus sentidos disponibles.

Ejemplo

Permitir que la aplicación completa se pueda navegar utilizando exclusivamente el teclado (mediante el uso ordenado de la tecla Tabulador), algo vital para usuarios con discapacidades motoras que no pueden manipular un ratón o una pantalla táctil.

2. Operable (Operable):

Los componentes interactivos de la interfaz y la navegación deben poder ser manejados por cualquier usuario, independientemente del periférico que utilicen.

Ejemplo

Permitir que la aplicación completa se pueda navegar utilizando exclusivamente el teclado (mediante el uso ordenado de la tecla Tabulador), algo vital para usuarios con discapacidades motoras que no pueden manipular un ratón o una pantalla táctil.

3. Comprensible (Understandable):

El funcionamiento de la aplicación, los textos y la lógica de navegación deben ser claros, predecibles y fáciles de entender para evitar la confusión cognitiva.

Ejemplo

Mantener la consistencia en los menús de todas las pantallas para que el usuario no tenga que reaprender a orientarse en cada sección, o estructurar los formularios con etiquetas claras y directrices comprensibles.

4. Robusto (Robust):

El contenido y los componentes de la interfaz deben ser lo suficientemente fiables como para ser interpretados de manera correcta por una amplia variedad de tecnologías, incluidos los navegadores actuales, dispositivos antiguos y, sobre todo, herramientas de asistencia especializadas.

Ejemplo

Utilizar componentes de software y etiquetas estándar (HTML semántico o controles nativos de UI) que no fallen al ser leídos por software de asistencia para personas ciegas.

3. Arquitectura de la información

Una vez comprendidos los principios de usabilidad y accesibilidad, el proceso de prototipado de una interfaz exige abordar su estructura interna. Antes de definir colores, tipografías o botones concretos, es necesario organizar qué información contendrá la aplicación y cómo se moverá el usuario a través de ella. Esta fase se conoce como Arquitectura de la Información (AI) y diseño de la navegación.

La arquitectura de la información se encarga de estructurar, clasificar y etiquetar los contenidos de un sistema informático para que los usuarios puedan encontrar la información y completar sus tareas con el mínimo esfuerzo.

Ejemplo

Imagina entrar a un gran supermercado donde los productos están colocados al azar: la leche junto a las herramientas de ferretería y los medicamentos en la sección de frutería. El resultado sería el caos y la frustración. Una buena Arquitectura de la Información funciona exactamente igual que la señalización y distribución lógica de un establecimiento físico: permite que cualquier cliente se oriente de forma intuitiva desde el primer segundo sin necesidad de preguntar dónde está cada cosa.

En el contexto del desarrollo de aplicaciones, diseñar una buena arquitectura implica:

- **Jerarquizar los contenidos:** Ordenar la información estableciendo prioridades entre lo fundamental y lo secundario, guiando al usuario de lo general a lo específico según sus necesidades de uso.
- **Reducir la carga cognitiva:** Minimizar el esfuerzo mental y la memorización que el sistema exige al usuario, evitando que tenga que adivinar dónde se localizan las distintas funciones o descifrar menús confusos.

Ejemplos

- Para la jerarquización: En una aplicación de banca móvil, la consulta de saldo y la transferencia rápida deben estar accesibles de manera inmediata en la pantalla de inicio (primer nivel), mientras que los extractos de contratos antiguos o la descarga de normativas legales se relegan a un segundo o tercer nivel de profundidad.
- Para la reducción de la carga cognitiva: Si una tienda online oculta el botón de "Carrito de la compra" dentro de un menú secundario de ajustes, se obliga al usuario a realizar un esfuerzo mental innecesario rompiendo los estándares lógicos del sector.

3.1 Estructuras de navegación

Una vez organizada la arquitectura de la información, el siguiente paso consiste en definir los caminos que permitirán al usuario desplazarse entre las diferentes pantallas. El puente visual entre la arquitectura conceptual y el diseño de pantallas es el mapa del sitio (sitemap) o diagrama de árbol, un esquema que plasma gráficamente cómo se interconectan los distintos contenidos y niveles de la aplicación.

A partir de este mapa, se definen los tipos de estructuras de navegación principales en el desarrollo de software:

- **Estructura Lineal:** Las pantallas se suceden en un orden secuencial estricto, donde el usuario avanza o retrocede paso a paso sin posibilidad de saltar secciones intermedias.
- **Estructura Jerárquica (en Árbol):** Es la más habitual en aplicaciones complejas. Los contenidos se organizan por niveles de importancia partiendo de una pantalla principal (Home o panel de control) de la cual cuelgan categorías y subcategorías. El usuario profundiza para detallar la información y utiliza los mecanismos de retorno para subir de nivel.
- **Estructura en Red o Matricial:** Permite al usuario saltar de forma libre y transversal entre diferentes contenidos que guardan relación entre sí, sin obligarle a pasar por una jerarquía estricta.

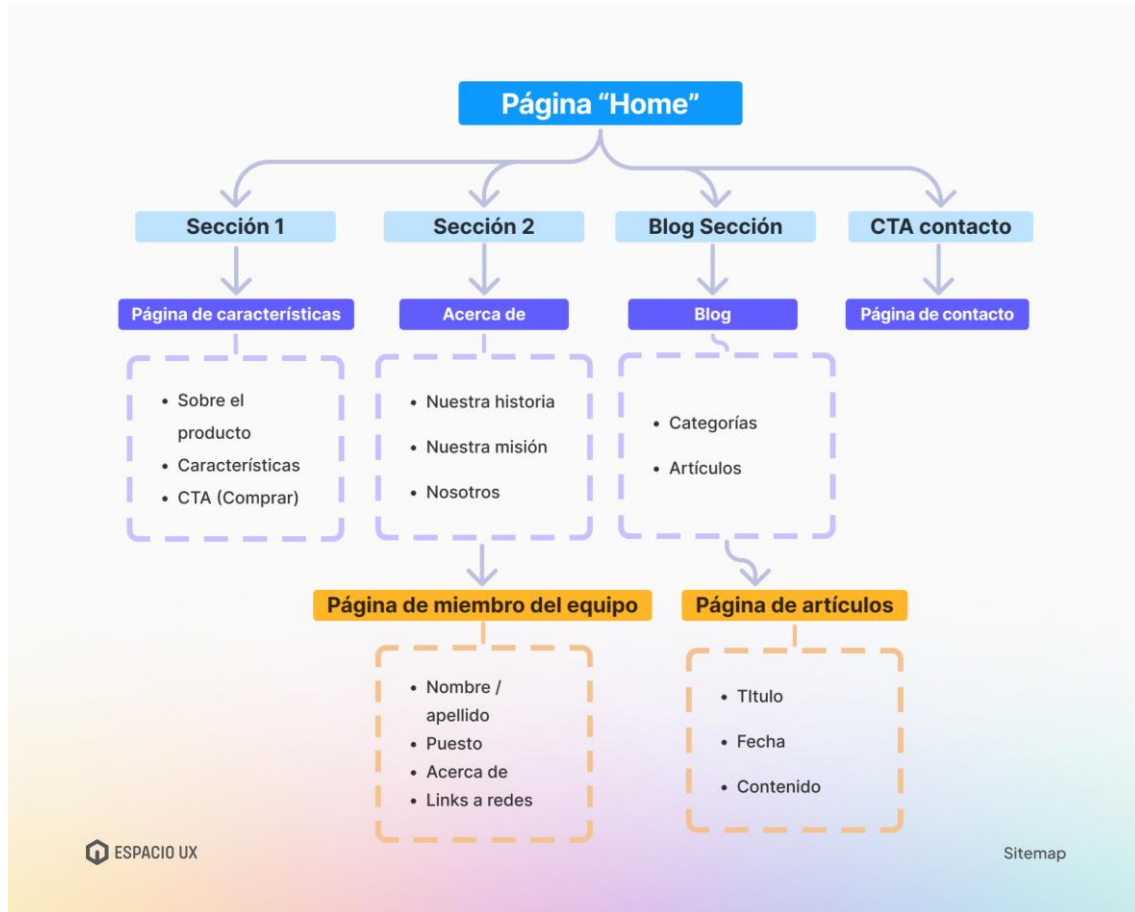
Aplicaciones típicas

- **Estructura Lineal:** Asistentes de configuración (wizards) o procesos críticos de pago en comercio electrónico (Paso 1: Dirección -> Paso 2: Envío -> Paso 3: Pago).
- **Estructura Jerárquica:** Paneles de administración, aplicaciones bancarias o plataformas de e-learning.
- **Estructura en Red:** Enciclopedias digitales, blogs o software de diseño con múltiples herramientas interconectadas.

3.2 Mapa del Sitio (Sitemap)

El mapa del sitio es un esquema visual en forma de diagrama de árbol que representa de manera jerárquica todas las pantallas, páginas o secciones de una aplicación y las conexiones que existen entre ellas.

Actúa como el plano arquitectónico de un edificio antes de empezar a construir. Permite detectar si falta alguna sección, si hay pantallas aisladas sin salida o si la profundidad de los clics es excesiva (obligando al usuario a navegar demasiados niveles hacia abajo).



3.2 Patrones de Menús

Para materializar estas estructuras en la interfaz gráfica, se emplean patrones de menús estandarizados que el usuario reconoce de manera intuitiva:

- **Barra de navegación superior (Top Bar):** Ubicada en la parte alta de la pantalla. Es el estándar en aplicaciones web y de escritorio para mostrar el logotipo, la búsqueda global y accesos directos al perfil o notificaciones.
- **Barra de navegación lateral (Sidebar):** Panel vertical persistente, muy común en aplicaciones de escritorio o dashboards complejos. Permite albergar una gran cantidad de opciones de primer nivel sin colapsar el espacio central de trabajo.
- **Menú desplegable u oculto (Menú Hamburguesa - Hamburger Menu):** Oculta las opciones secundarias tras un icono de tres líneas

horizontales. Muy utilizado en aplicaciones móviles para ahorrar espacio en pantallas reducidas, aunque debe reservarse para contenidos de menor frecuencia de uso para no ocultar la navegación principal.

- **Barra de navegación inferior (Bottom Navigation):** El estándar absoluto en aplicaciones móviles nativas. Sitúa entre 3 y 5 accesos directos a las secciones principales de la app en la parte inferior, justo en la "zona de confort" del pulgar para facilitar el uso con una sola mano.

Enlaces de interés:

<https://www.diegoamorin.com/10-principios-usabilidad/>

<https://www.espacioux.com/blog/sitemap-ux#anchor-1>